Федеральное государственное автономное образовательное учреждение высшего образования
«Национальный исследовательский ядерный университет „МИФИ“»

ОТЧЁТ
по лабораторной работе № 3
по дисциплине «Криптография»

Создание электронной подписи в документе

Вариант № 7

Выполнил: студент 1-го курса магистратуры
Гафт Илья Маратович

Преподаватель: Епишкина Анна Васильевна

Москва — 2026

1. Цель и постановка задачи

Цель работы — разработать процедуры формирования и проверки электронной подписи сообщения на основе асимметричного алгоритма с использованием контрольной суммы, заданной вариантом.

Задачи: реализовать вычисление контрольной суммы, формирование и проверку подписи; продемонстрировать успешную проверку исходного сообщения и отказ при изменении сообщения либо подписи; выполнить автоматические тесты.

2. Исходные данные и вариант

Номер в списке группы: 30. Количество вариантов: 12. Номер варианта: (30 mod 12) + 1 = 7.

Вариант 7: сообщение кодируется в UTF-8, разбивается на 16-битные блоки; при обработке каждого блока выполняются XOR с накопленным значением и циклический сдвиг полученного 16-битного слова влево на один разряд. В реализации используется порядок байтов big-endian, неполный последний блок дополняется нулевым байтом справа. Эти соглашения фиксируют неоднозначности исходной формулировки.

Схема подписи основана на описанном в методичке алгоритме ГОСТ Р 34.10-94. Для демонстрации используются фиксированные параметры p (512 бит), q (256 бит), a; параметры удовлетворяют q | (p−1), 1 < a < p−1, a^q mod p = 1. Простота p и q проверяется вероятностным тестом Миллера—Рабина (40 раундов).

p =
10384704642531197664116868987034252164839301013025827535259107039537005568296882902039420609371913725819647695675081014375901705793400586023922425046793447

q =
111010004376550013782823712650965252593978046313480548030881300191926658557793

a =
376794328774378899180351664782758136435445503455533509039125705045832252116637734708373526480345429425295642269383176949471899556117005532542781202813270 8

3. Алгоритм работы программы

Контрольная сумма: $C_0 = 0$; для каждого 16-битного блока B_i вычисляется $C_i = ROL16(C_{i-1} XOR B_i, 1)$. Для подписи используется h = C mod q; если h = 0, полагается h = 1.

Ключи: закрытый ключ x выбирается в диапазоне $1 \le x < q$; открытый ключ y = a^x mod p. Для подписи случайно выбирается одноразовый k ($1 \le k < q$), вычисляются r = (a^k mod p) mod q и s = (xr + kh) mod q. Если r = 0 или s = 0, выбор k повторяется.

Проверка: проверяются $0 < r,s < q$ и допустимость открытого ключа. Вычисляются v = h^(q−2) mod q, z_1 = sv mod q, z_2 = (q−r)v mod q, u = (($a^{z_1} \cdot y^{z_2}$ mod p) mod q). Подпись принимается тогда и только тогда, когда u = r.

Одноразовый параметр k формируется криптографическим генератором secrets. Закрытый ключ также формируется через secrets. Программа реализована на Python 3; вычисления степеней по модулю выполняются встроенной функцией pow.

4. Структура программы

src/checksum.py — контрольная сумма варианта №7; src/gost_signature.py — параметры, ключи, подпись и проверка; src/main.py — демонстрационный запуск; tests/test_checksum.py и tests/test_signature.py — автоматические тесты.

5. Результаты выполнения

На представленном снимке демонстрационного запуска исходное сообщение «Электронная подпись документа» имеет контрольную сумму 0974 (HEX), или 2420 (DEC). Проверка исходной подписи завершилась сообщением «ПОДПИСЬ ВЕРНА». Для изменённого сообщения «Электронная подпись документа!» и для изменённой подписи получен результат «ПОДПИСЬ НЕВЕРНА».

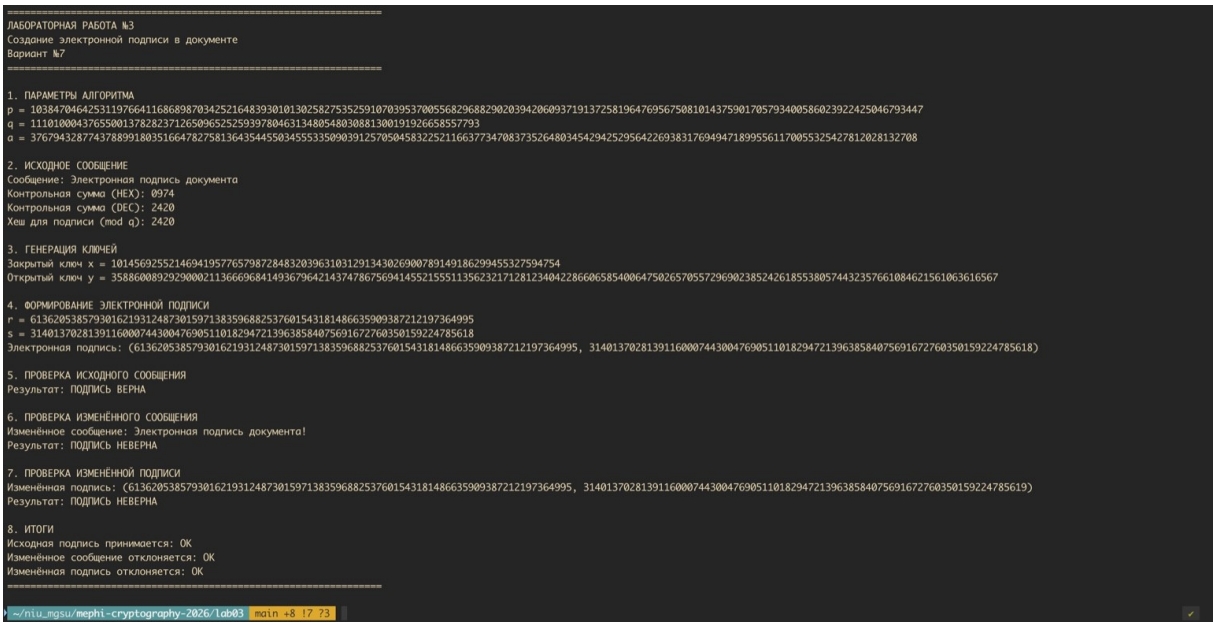


Рисунок 1 — Формирование подписи и проверка трёх сценариев

На предоставленном снимке автоматического тестирования показано: Ran 29 tests in 0.051s; OK. Таким образом, на данном запуске 29 тестов завершились успешно. В отчёте приведены именно результаты предоставленного запуска, без утверждений о дополнительном независимом прогоне.

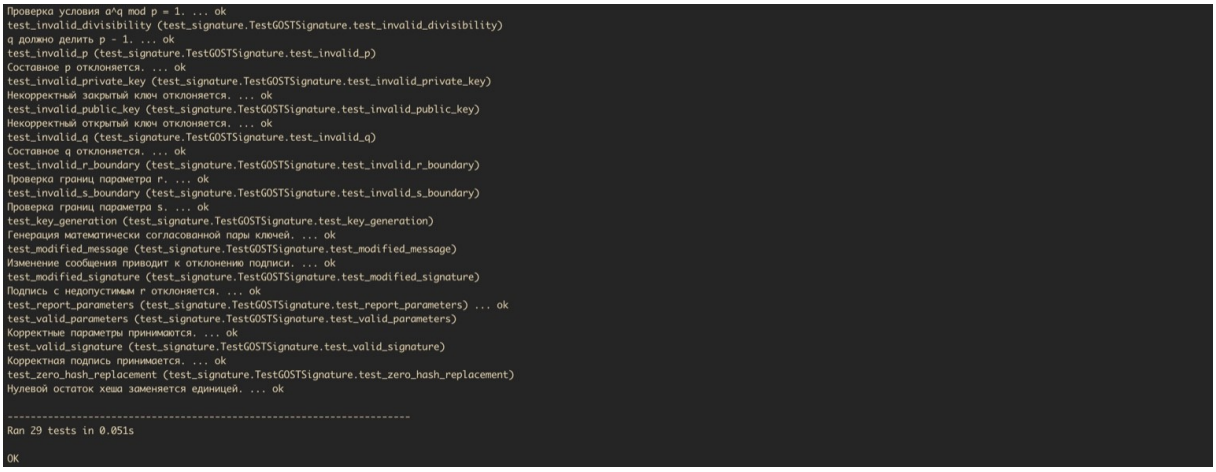


Рисунок 2 — Результат запуска набора из 29 тестов

6. Анализ результатов

Корректная подпись прошла проверку. Изменение сообщения и изменение одного из компонентов подписи в представленных примерах привели к отклонению. Тестовый набор проверяет корректные и некорректные параметры, генерацию ключей, граничные значения подписи и специальную обработку нулевого остатка контрольной суммы.

Ограничение: 16-битная контрольная сумма имеет не более 65 536 значений, допускает коллизии и не обладает требуемой криптографической стойкостью. Поэтому изменения не гарантированно обнаруживаются: два разных сообщения с одинаковой контрольной суммой могут пройти проверку с одной подписью. Применение этой суммы обусловлено учебным вариантом; реализация не является полноценной стандартной защищённой системой ГОСТ Р 34.10-94, где предусмотрена соответствующая криптографическая хеш-функция.

7. Ответы на контрольные вопросы

1. Какие криптоалгоритмы используются для создания электронной подписи?

Асимметричные схемы, например RSA, DSA, ECDSA и ГОСТ Р 34.10; перед подписанием обычно вычисляется криптографический хеш сообщения.

2. Что такое криптографическая хеш-функция, какими свойствами она должна обладать?

Функция, отображающая сообщения произвольной длины в дайджест фиксированной длины. Для применения в подписи важны устойчивость к нахождению прообраза, второго прообраза и коллизий.

3. Как содержание сообщения влияет на электронную подпись?

Подпись вычисляется с использованием хеша сообщения. Изменение содержимого обычно меняет хеш и нарушает равенство проверки; при коллизии хеша это не гарантировано.

4. Где используется ЭП?

В электронном документообороте, подписании договоров, отчётности, программных пакетов и электронных транзакций.

5. В каком случае электронная подпись при проверке отвергается?

Когда r или s вне допустимого диапазона, ключ недопустим либо вычисленное значение u не равно r.

6. От каких угроз информации защищает ЭП?

При корректной криптографической реализации обеспечивает проверку целостности, подлинности происхождения и доказательство владения ключом подписания; не обеспечивает конфиденциальность.

7. Какие условия должны соблюдаться для параметра a в системе ЭП?

$1 < a < p-1$ и $a^q \bmod p = 1$; вместе с простотой q эти условия задают элемент нужной подгруппы.

8. Как открытый ключ y связан с секретным ключом x?

$y = a^x \bmod p$.

9. Почему контрольные суммы (например, CRC32) не подходят для использования в схемах ЭП?

Они предназначены для обнаружения случайных ошибок и позволяют относительно легко строить коллизии; криптографической стойкости нет.

10. Как функция хеширования связана с защитой от изменений документа?

Подписывается значение, вычисленное из содержимого документа; при изменении данных обычно меняется хеш, поэтому прежняя подпись перестаёт подходить.

11. Почему при внесении изменений в подписанное сообщение подпись становится недействительной?

Проверка использует хеш полученного сообщения вместо исходного. Если хеш изменился, проверочное равенство обычно не выполняется. При коллизии это утверждение не гарантировано.

12. Почему в алгоритме ГОСТ Р34.10-94 длина подписи составляет 512 бит?

Подпись состоит из двух компонентов r и s, каждый кодируется 256 битами при 256-битном q: 256 + 256 = 512 бит.

13. Какие шаги необходимо выполнить, чтобы подделать электронную подпись, и почему это вычислительно сложно?

Злоумышленнику нужно получить корректную пару (r,s) для сообщения без закрытого ключа. Для правильно выбранных параметров и стойкого хеша это связано со сложностью задачи дискретного логарифмирования и невозможностью эффективно обойти криптографическую хеш-функцию. В данной учебной версии слабая сумма существенно снижает защиту.

14. Почему размеры параметров p и q строго регламентированы в ГОСТ?

Они определяют размер подгруппы, вычислительную сложность атак и совместимость реализаций; слишком малые значения делают подбор и решение дискретного логарифма доступнее.

15. Почему в алгоритме требуется, чтобы параметр q был простым делителем p−1?

Мультипликативная группа по модулю простого p имеет порядок p−1. Условие q | (p−1) позволяет выбрать подгруппу порядка q; простота q обеспечивает существование обратного элемента для любого ненулевого хеша по модулю q.

8. Выводы

Реализована учебная схема формирования и проверки электронной подписи по алгоритму ГОСТ Р 34.10-94 с контрольной суммой варианта №7. На представленном демонстрационном запуске подтверждена корректная проверка исходного сообщения и отклонение двух конкретных модификаций. Представленный результат тестирования: 29 тестов, статус OK. Ограничение работы — низкая стойкость 16-битной контрольной суммы; полученная система предназначена для изучения алгоритма, а не для практической защиты документов.

Приложение А. Исходный код программы

src/checksum.py

```
"""
Лабораторная работа №3.
Вариант №7.


Вычисление 16-битной контрольной суммы:
- разбиение сообщения на блоки по 16 бит;
- XOR каждого блока с накопленной суммой;
- циклический сдвиг результата влево на 1 бит.
"""


MASK_16 = 0xFFFF



def rotate_left_16(value: int) -> int:
    """Циклический сдвиг 16-битного числа влево на 1 бит."""
    value &= MASK_16
    return ((value << 1) | (value >> 15)) & MASK_16



def checksum_16(message: str) -> int:
    """
    Вычисляет контрольную сумму сообщения по варианту №7.

    Кодировка: UTF-8.
    Порядок байтов: big-endian.
    Неполный последний блок дополняется нулями справа.
    Для каждого блока: checksum = ROL16(checksum XOR block).

    Args:
```

```
        message: Исходное текстовое сообщение.

    Returns:
        Контрольная сумма в диапазоне 0..65535.
    """
    if not isinstance(message, str):
        raise TypeError("Сообщение должно быть строкой")

    data = message.encode("utf-8")

    if len(data) % 2 != 0:
        data += b"\x00"

    checksum = 0

    for i in range(0, len(data), 2):
        block = int.from_bytes(
            data[i:i + 2],
            byteorder="big"
        )

        checksum ^= block
        checksum = rotate_left_16(checksum)

    return checksum



def checksum_hex(message: str) -> str:
    """Возвращает контрольную сумму в шестнадцатеричном виде."""
    return f"{checksum_16(message):04X}"
```

src/gost_signature.py

```
"""
Лабораторная работа №3.
Учебная реализация электронной подписи ГОСТ Р 34.10-94.

В качестве функции хеширования используется
контрольная сумма варианта №7.
"""

import secrets
from functools import lru_cache
from dataclasses import dataclass

from src.checksum import checksum_16



@dataclass(frozen=True)
class GOSTParameters:
    p: int
    q: int
    a: int



@dataclass(frozen=True)
class GOSTKeyPair:
    private_key: int
    public_key: int
```

```
@dataclass(frozen=True)
class GOSTSignature:
    r: int
    s: int



def is_prime(n: int, rounds: int = 40) -> bool:
    """Вероятностная проверка простоты Миллера–Рабина (40 раундов)."""
    if n < 2:
        return False
    small = (2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37)
    for prime in small:
        if n == prime:
            return True
        if n % prime == 0:
            return False
    d, r = n - 1, 0
    while d % 2 == 0:
        d //= 2
        r += 1
    for _ in range(rounds):
        base = secrets.randbelow(n - 3) + 2
        x = pow(base, d, n)
        if x in (1, n - 1):
            continue
        for _ in range(r - 1):
            x = pow(x, 2, n)
            if x == n - 1:
                break
        else:
            return False
    return True



@lru_cache(maxsize=32)
def validate_parameters(params: GOSTParameters) -> None:
    """Проверяет простоту и групповые условия; результаты кешируются."""
    p, q, a = params.p, params.q, params.a
    if not is_prime(p):
        raise ValueError("Параметр p должен быть простым")
    if not is_prime(q):
        raise ValueError("Параметр q должен быть простым")
    if (p - 1) % q != 0:
        raise ValueError("q должен делить p - 1")
    if not (1 < a < p - 1):
        raise ValueError("Должно выполняться 1 < a < p - 1")
    if pow(a, q, p) != 1:
        raise ValueError("Должно выполняться a^q mod p = 1")



def generate_keys(params: GOSTParameters) -> GOSTKeyPair:
    """Генерация закрытого и открытого ключей."""
    validate_parameters(params)


    x = secrets.randbelow(params.q - 1) + 1
    y = pow(params.a, x, params.p)


    return GOSTKeyPair(
        private_key=x,
        public_key=y,
    )



def message_hash(message: str, q: int) -> int:
```

```
    """Вычисление хеша сообщения для алгоритма подписи."""
    h = checksum_16(message) % q

    return h if h != 0 else 1



def sign_message(
    message: str,
    private_key: int,
    params: GOSTParameters,
) -> GOSTSignature:
    """Формирование электронной подписи (r, s)."""
    validate_parameters(params)

    p, q, a = params.p, params.q, params.a

    if not (1 <= private_key < q):
        raise ValueError("Некорректный закрытый ключ")

    h = message_hash(message, q)

    while True:
        k = secrets.randbelow(q - 1) + 1

        r = pow(a, k, p) % q

        if r == 0:
            continue

        s = (private_key * r + k * h) % q

        if s == 0:
            continue

        return GOSTSignature(r=r, s=s)



def verify_signature(
    message: str,
    signature: GOSTSignature,
    public_key: int,
    params: GOSTParameters,
) -> bool:
    """Проверка электронной подписи по открытому ключу."""
    validate_parameters(params)

    p, q, a = params.p, params.q, params.a

    if not (0 < signature.r < q):
        return False

    if not (0 < signature.s < q):
        return False

    if not (1 < public_key < p):
        return False

    if pow(public_key, q, p) != 1:
        return False
```

```
    h = message_hash(message, q)

    v = pow(h, q - 2, q)

    z1 = (signature.s * v) % q
    z2 = ((q - signature.r) * v) % q

    u = (
        pow(a, z1, p) *
        pow(public_key, z2, p)
    ) % p % q

    return u == signature.r



def get_demo_parameters() -> GOSTParameters:
    """
    Небольшие учебные параметры.

    p = 23
    q = 11
    a = 2

    11 делит 22.
    2^11 mod 23 = 1.

    Не предназначены для реальной криптографии.
    """
    params = GOSTParameters(
        p=23,
        q=11,
        a=2,
    )

    validate_parameters(params)

    return params



def get_report_parameters() -> GOSTParameters:
    """Фиксированные параметры с p=512 бит и q=256 бит для лабораторной."""
    params = GOSTParameters(

p=103847046425311976641168689870342521648393010130258275352591070395370055682968829020394206093719137258196476956750810143759017057934005860239224250467934 47,
        q=111010004376550013782823712650965252593978046313480548030881300191926658557793,

a=37679432877437889918035166478275813643544550345553350903912570504583225211663773470837352648034542942529564226938317694947189955611700553254278120281327 08,
    )
    validate_parameters(params)
    return params
```

src/main.py

```
"""
Лабораторная работа №3.
Создание электронной подписи в документе.


Вариант №7.
ГОСТ Р 34.10-94 (учебная реализация).
```

```
"""

from src.checksum import checksum_16, checksum_hex
from src.gost_signature import (
    GOSTSignature,
    generate_keys,
    get_report_parameters,
    message_hash,
    sign_message,
    verify_signature,
)



def print_separator():
    print("=" * 65)



def demonstrate(message: str):
    """Демонстрация формирования и проверки электронной подписи."""


    print_separator()
    print("ЛАБОРАТОРНАЯ РАБОТА №3")
    print("Создание электронной подписи в документе")
    print("Вариант №7")
    print_separator()


    # 1. Параметры алгоритма
    params = get_report_parameters()


    print("\n1. ПАРАМЕТРЫ АЛГОРИТМА")
    print(f"p = {params.p}")
    print(f"q = {params.q}")
    print(f"a = {params.a}")


    # 2. Исходное сообщение и контрольная сумма
    print("\n2. ИСХОДНОЕ СООБЩЕНИЕ")
    print(f"Сообщение: {message}")
    print(f"Контрольная сумма (HEX): {checksum_hex(message)}")
    print(f"Контрольная сумма (DEC): {checksum_16(message)}")
    print(f"Хеш для подписи (mod q): {message_hash(message, params.q)}")


    # 3. Генерация ключей
    keys = generate_keys(params)


    print("\n3. ГЕНЕРАЦИЯ КЛЮЧЕЙ")
    print(f"Закрытый ключ x = {keys.private_key}")
    print(f"Открытый ключ y = {keys.public_key}")


    # 4. Формирование подписи
    signature = sign_message(
        message,
        keys.private_key,
        params,
    )


    print("\n4. ФОРМИРОВАНИЕ ЭЛЕКТРОННОЙ ПОДПИСИ")
    print(f"r = {signature.r}")
    print(f"s = {signature.s}")
    print(f"Электронная подпись: ({signature.r}, {signature.s})")


    # 5. Проверка корректной подписи
    valid = verify_signature(
```

```
    message,
    signature,
    keys.public_key,
    params,
)


print("\n5. ПРОВЕРКА ИСХОДНОГО СООБЩЕНИЯ")
print(f"Результат: {'ПОДПИСЬ ВЕРНА' if valid else 'ПОДПИСЬ НЕВЕРНА'}")


# 6. Изменение сообщения
# Выбираем изменение, действительно меняющее 16-битную сумму.
modified_message = message + "!"
if message_hash(modified_message, params.q) == message_hash(message, params.q):
    raise RuntimeError("Изменение сообщения не изменило контрольную сумму")


modified_valid = verify_signature(
    modified_message,
    signature,
    keys.public_key,
    params,
)


print("\n6. ПРОВЕРКА ИЗМЕНЁННОГО СООБЩЕНИЯ")
print(f"Изменённое сообщение: {modified_message}")
print(
    "Результат:",
    "ПОДПИСЬ ВЕРНА" if modified_valid else "ПОДПИСЬ НЕВЕРНА",
)


# 7. Изменение подписи
# Изменяем s в допустимом диапазоне и ищем отвергаемый вариант.
# Это проверяет математическую верификацию, а не только границы.
modified_signature = None
for offset in range(1, 5):
    new_s = (signature.s - 1 + offset) % (params.q - 1) + 1
    if new_s == signature.s:
        continue
    candidate = GOSTSignature(r=signature.r, s=new_s)
    if not verify_signature(message, candidate, keys.public_key, params):
        modified_signature = candidate
        break
if modified_signature is None:
    raise RuntimeError("Не удалось получить отвергаемую изменённую подпись")


signature_valid = verify_signature(
    message,
    modified_signature,
    keys.public_key,
    params,
)


print("\n7. ПРОВЕРКА ИЗМЕНЁННОЙ ПОДПИСИ")
print(
    f"Изменённая подпись: "
    f"({modified_signature.r}, {modified_signature.s})"
)
print(
    "Результат:",
    "ПОДПИСЬ ВЕРНА" if signature_valid else "ПОДПИСЬ НЕВЕРНА",
)


# 8. Итоги
print("\n8. ИТОГИ")


checks = {
    "Исходная подпись принимается": valid,
```

```
        "Изменённое сообщение отклоняется": not modified_valid,
        "Изменённая подпись отклоняется": not signature_valid,
    }


    for description, result in checks.items():
        print(f"{description}: {'OK' if result else 'FAIL'}")


    print_separator()


    return all(checks.values())



if __name__ == "__main__":
    examples = [
        "AB",
        "Лабораторная работа №3",
        "Электронная подпись документа",
    ]


    for example in examples:
        if not demonstrate(example):
            raise SystemExit("Ошибка демонстрации")
        print()
```

tests/test_checksum.py

```
import unittest

from src.checksum import (
    rotate_left_16,
    checksum_16,
    checksum_hex,
)



class TestChecksum(unittest.TestCase):


    def test_rotate_left(self):
        """Проверка циклического сдвига."""
        self.assertEqual(rotate_left_16(0x8000), 0x0001)
        self.assertEqual(rotate_left_16(0x0001), 0x0002)
        self.assertEqual(rotate_left_16(0xFFFF), 0xFFFF)


    def test_empty_message(self):
        """Пустое сообщение."""
        self.assertEqual(checksum_16(""), 0)


    def test_single_block(self):
        """Один 16-битный блок: AB."""
        self.assertEqual(checksum_16("AB"), 0x8284)


    def test_multiple_blocks(self):
        """Два блока: AB и CD."""
        self.assertEqual(checksum_16("ABCD"), 0x8381)


    def test_odd_length(self):
        """Нечётное количество байтов."""
        self.assertEqual(checksum_16("A"), 0x8200)


    def test_different_messages(self):
        """Проверка контрольных сумм двух сообщений."""
        self.assertNotEqual(
```

```
            checksum_16("Hello"),
            checksum_16("World"),
        )

    def test_unicode(self):
        """Проверка корректной обработки UTF-8."""
        result = checksum_16("Привет")
        self.assertIsInstance(result, int)
        self.assertGreaterEqual(result, 0)
        self.assertLessEqual(result, 0xFFFF)

    def test_hex_format(self):
        """Проверка шестнадцатеричного представления."""
        self.assertEqual(checksum_hex("AB"), "8284")

    def test_invalid_input(self):
        """Проверка некорректного типа данных."""
        with self.assertRaises(TypeError):
            checksum_16(123)


if __name__ == "__main__":
    unittest.main()
```

tests/test_signature.py

```
"""Тестирование учебной реализации ГОСТ Р 34.10-94."""

import unittest
from unittest.mock import patch

from src.gost_signature import (
    GOSTParameters,
    GOSTSignature,
    generate_keys,
    get_demo_parameters,
    get_report_parameters,
    message_hash,
    sign_message,
    validate_parameters,
    verify_signature,
)


class TestGOSTSignature(unittest.TestCase):

    def setUp(self):
        self.params = get_demo_parameters()
        self.message = "Лабораторная работа №3"

        # Фиксированный ключ для воспроизводимости.
        self.private_key = 7
        self.public_key = pow(
            self.params.a,
            self.private_key,
            self.params.p,
        )

    def test_report_parameters(self):
        params = get_report_parameters()
        self.assertEqual(params.p.bit_length(), 512)
        self.assertEqual(params.q.bit_length(), 256)
        validate_parameters(params)
```

```
def test_in_range_tampering(self):
    params = get_report_parameters()
    x = 7
    y = pow(params.a, x, params.p)
    sig = sign_message("AB", x, params)
    self.assertTrue(verify_signature("AB", sig, y, params))
    self.assertFalse(verify_signature("AC", sig, y, params))
    altered = GOSTSignature(sig.r, sig.s % (params.q - 1) + 1)
    self.assertTrue(0 < altered.s < params.q)
    self.assertFalse(verify_signature("AB", altered, y, params))


def test_checksum_collision(self):
    # XOR-свёртка не обладает криптографической стойкостью.
    from src.checksum import checksum_16
    self.assertEqual(checksum_16(""), checksum_16("\x00\x00"))


def test_collision_keeps_signature_valid(self):
    """Доказывает ограничение 16-битной суммы: коллизия проходит проверку."""
    params = get_report_parameters()
    x = 7
    y = pow(params.a, x, params.p)
    signature = sign_message("", x, params)
    self.assertTrue(verify_signature("", signature, y, params))
    self.assertTrue(verify_signature("\x00\x00", signature, y, params))


def test_valid_parameters(self):
    """Корректные параметры принимаются."""
    self.assertIsNone(
        validate_parameters(self.params)
    )


def test_invalid_p(self):
    """Составное p отклоняется."""
    with self.assertRaises(ValueError):
        validate_parameters(
            GOSTParameters(p=21, q=11, a=2)
        )


def test_invalid_q(self):
    """Составное q отклоняется."""
    with self.assertRaises(ValueError):
        validate_parameters(
            GOSTParameters(p=23, q=4, a=2)
        )


def test_invalid_divisibility(self):
    """q должно делить p - 1."""
    with self.assertRaises(ValueError):
        validate_parameters(
            GOSTParameters(p=23, q=5, a=2)
        )


def test_invalid_a(self):
    """Проверка условия a^q mod p = 1."""
    with self.assertRaises(ValueError):
        validate_parameters(
            GOSTParameters(p=23, q=11, a=5)
        )


def test_key_generation(self):
    """Генерация математически согласованной пары ключей."""
    keys = generate_keys(self.params)


    self.assertTrue(
```

```
        1 <= keys.private_key < self.params.q
    )

    self.assertEqual(
        keys.public_key,
        pow(
            self.params.a,
            keys.private_key,
            self.params.p,
        ),
    )


def test_valid_signature(self):
    """Корректная подпись принимается."""
    signature = sign_message(
        self.message,
        self.private_key,
        self.params,
    )

    self.assertTrue(
        verify_signature(
            self.message,
            signature,
            self.public_key,
            self.params,
        )
    )


def test_modified_message(self):
    """Изменённое сообщение отклоняется в выбранном примере."""
    signature = sign_message(
        "AB",
        self.private_key,
        self.params,
    )

    self.assertFalse(
        verify_signature(
            "AC",
            signature,
            self.public_key,
            self.params,
        )
    )


def test_modified_signature(self):
    """Подпись с недопустимым r отклоняется."""
    signature = GOSTSignature(r=0, s=5)

    self.assertFalse(
        verify_signature(
            self.message,
            signature,
            self.public_key,
            self.params,
        )
    )


def test_invalid_r_boundary(self):
    """Проверка границ параметра r."""
    for r in (0, self.params.q, -1):
        with self.subTest(r=r):
            self.assertFalse(
                verify_signature(
                    self.message,
                    GOSTSignature(r=r, s=1),
```

```
                    self.public_key,
                    self.params,
                )
            )


def test_invalid_s_boundary(self):
    """Проверка границ параметра s."""
    for s in (0, self.params.q, -1):
        with self.subTest(s=s):
            self.assertFalse(
                verify_signature(
                    self.message,
                    GOSTSignature(r=1, s=s),
                    self.public_key,
                    self.params,
                )
            )


def test_invalid_public_key(self):
    """Некорректный открытый ключ отклоняется."""
    signature = sign_message(
        self.message,
        self.private_key,
        self.params,
    )


    self.assertFalse(
        verify_signature(
            self.message,
            signature,
            0,
            self.params,
        )
    )


def test_hash_range(self):
    """Хеш приводится к диапазону 1..q-1."""
    h = message_hash(self.message, self.params.q)


    self.assertTrue(1 <= h < self.params.q)


def test_zero_hash_replacement(self):
    """Нулевой остаток хеша заменяется единицей."""
    self.assertEqual(message_hash("", self.params.q), 1)


def test_invalid_private_key(self):
    """Некорректный закрытый ключ отклоняется."""
    with self.assertRaises(ValueError):
        sign_message(
            self.message,
            0,
            self.params,
        )


def test_deterministic_signature(self):
    """
    Проверка подписи при фиксированном k.


    Сообщение AB:
    checksum = 0x8284 = 33412
    h = 33412 mod 11 = 5


    x = 7, k = 3
    r = (2^3 mod 23) mod 11 = 8
    s = (7*8 + 3*5) mod 11 = 5
```

```
        """
        with patch(
            "src.gost_signature.secrets.randbelow",
            return_value=2,
        ):
            signature = sign_message(
                "AB",
                self.private_key,
                self.params,
            )

        self.assertEqual(
            signature,
            GOSTSignature(r=8, s=5),
        )

        self.assertTrue(
            verify_signature(
                "AB",
                signature,
                self.public_key,
                self.params,
            )
        )



if __name__ == "__main__":
    unittest.main()
```